# Automating the SPS Translation Workflow

Alan Tipper

July 22, 2026

# Contents

# Chapter 1

# INTRODUCTION

The Translation workflow follows the translation industry ISO 18587 standard sequence of style guide generation, baseline translation, bilingual review and Line editing. This can be implemented using AI prompts and templates supplied with James Blatch's SPS "A.I TRANSLATIONS FOR AUTHORS course.

Whilst fully functional this chapter by chapter workflow can be quite time consuming with multiple document editing, cut and paste operations and document saving. With a typical 60,000 word novel split into 1500 word chapters this can result in a lot of manual file manipulation (e.g each operation is repeated 120 times for 40 chapters x 3 passes). Additionally the finite token limits of current AI web platforms puts a further bottleneck in this flow, It is not unusual to spend many days to get the baseline translation, error checking and line editing done before any manual proofing.

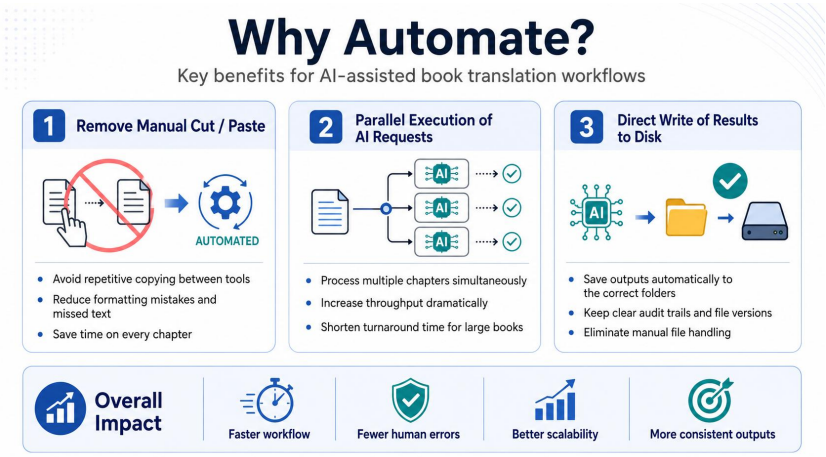


Figure 1.1: Reasons for Automating

As illustrated in figure 1.1 automation can enable full traceability, concurrent processing and automatic saving of intermediate documents. This can deliver considerable savings in time, cost and robustness of the process.

There are two main approaches to automating the workflow as shown in figure 1.2 (1) Concurrent API and (2) Multiple Agentic Loop within the AI.

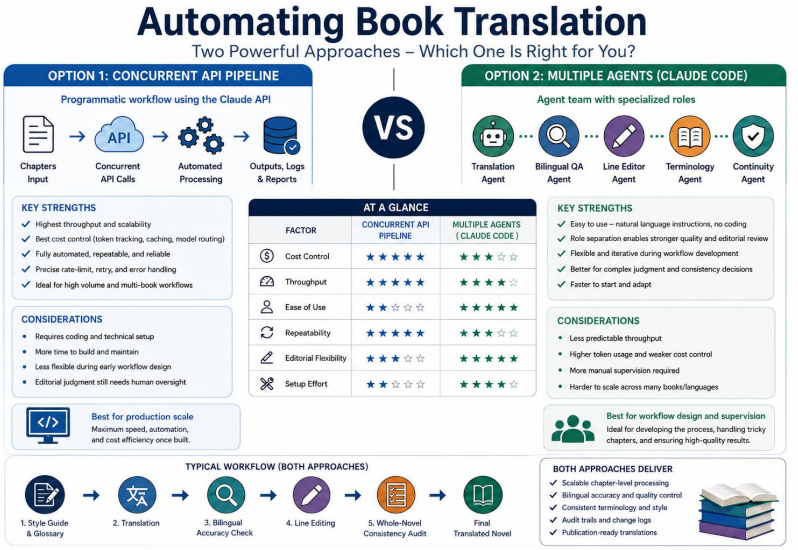


Figure 1.2: The Two Approaches

This document concentrates on option (1) as it is the more scalable and production ready solution. It is implemented using the AI SDK, a multi AI abstraction layer from liteLLM, python scripts to manage concurrent tasks via asyncio, python scripts to manage pre and post processing and a command shell to sequence the various tasks. This is shown below in figure 1.3

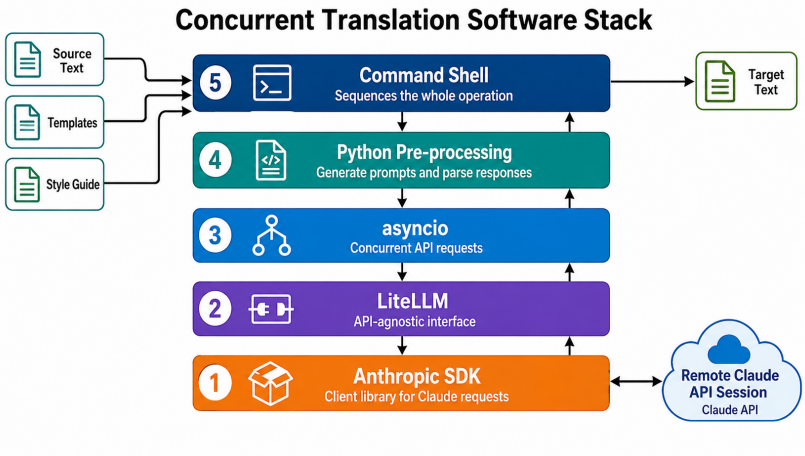


Figure 1.3: Software Stack

The stack operation can be broken down to multiple stages.

1. Automate the generation of per chapter translation prompt files from the manuscript and style guide

2. Use the AI API to submit multiple consecutive prompts for execution

3. Collate AI responses in individual files
4. Automatically parse the responses to separate translation text from notes
5. Merge the individual chapter files into a single document
6. Automate the generation of per chapter accuracy checking prompt files from the translation stage, the manuscript and the style guide
7. Use the AI API to submit multiple consecutive prompts for execution
8. Collate AI responses in individual files
9. Automatically parse the responses to separate checked text from notes
10. Merge the individual chapter files into a single document
11. Automate the generation of per chapter line editing prompt files from the accuracy check stage and the style guide
12. Use the AI API to submit multiple consecutive prompts for execution
13. Collate AI responses in individual files
14. Automatically parse the responses to separate edited text from notes
15. Merge the individual chapter files into a single line edited document

By maintaining the same overall flow as per the manual process and keeping all per chapter responses available it is possible to still use the web interface for moderate time savings or implement the full API method for big reductions in execution time. Additionally the individual chapters can be re-edited using the AI web tool if necessary and re-merged back into the final document(s).

Using the API rather than the web interface, whilst introducing extra complexity, brings a number of advantages

1. The API is a pay per use service that does not have the hard limits of the web max tokens limit
2. Whilst the API does have token rate limits they are easily managed.
3. The API can issue consecutive requests without having to wait for the AI responses hence significantly saving time
4. The API calls can be placed in a software loop with a small dwell between requests to automate an entire book submission

As an example of the huge savings from the API use the line edited translation of an 80,000 word novel went from a 10 day exercise using the original workflow to a 20 minute fully automated process costing £9 i.e less than the monthly subscription to the web interface and considerably less than commercial turn-key tools like Scribe Shadow.

This document describes the implementation of the automation to use the Claude API using Python and Bash scripts running on a Linux PC. They can be adapted for use in a Windows environment if required but will require the installation of Python (it is free!) and modification of the final automation

script to use CMD rather than Bash. The scripts can be easily adapted for other AI platforms e.g Gemini or Chat-GPT. In order to use the API a console account must be set up with Anthropic (or Chat-GPT/Google) and an API key generated to enable them to charge to a credit card.

## 1.1 OPEN SOURCE LICENSE

This software is supplied under the GNU General Public License, For full details see the LICENSE file. For clarity note that there is no warranty of any kind.

## 1.2 INSTALLING PREREQUISITS

The process requires a python interpreter and also the litellm and dotenv libraries to be installed. It also requires pandoc to be installed. These are all opensource tools so there is no cost associated with their use.

## 1.3 SETTING UP THE WORKSPACE

Download the zip file and extract it to a convenient location such as the desktop. Add your API key to the .env file in the top level subdirectory using the syntax "ANTHROPIC_API_KEY=your specific key" The zip file should extract to a set of sub directories as shown in 1.4


Figure 1.4: File Structure

## 1.4 RUNNING THE AUTOMATION SCRIPT

Before running the bash script add your style guide to the 01Style folder and call it styleguide.txt then add your translation_template.txt, Acccheck_template.txt and LineEdit_template.txt files to the templates folder. Finally add the markdown manuscript to be translated to the manuscript folder.

Make sure that the template files preserve the ”PROJECT STYLE SHEET” and ”CHAPTER START” lines as they will be used as insertion keys to add the style sheet and chapter text during prompt generation.

Determine the number of consecutive chapters to submit and the number of cycles of groups of chapters to complete. the product of cycles x chapters should be greater than or equal to the total chapter count. A maximum consecutive count of 6 has been found to work well. Edit the Bash/CMD script lines: python scripts/multiprompt_async.py ”translate_prompt_file_ch” ”AI_response” ”02Translate” 2 2

python scripts/multiprompt_async.py ”acc_check_prompt_file_ch” ”AI_response” ”03Check” 2 2

python scripts/multiprompt_async.py ”line_edit_prompt_file_ch” ”AI_response” ”04Edit” 2 2

For example for a 40 chapter book with 6 consecutive chapters per AI submission use python scripts/multiprompt_async.py ”translate_prompt_file_ch” ”AI_response” ”02Translate” 6 7

On Linux start the translation by using the bash command: ./startup.sh from the top level folder.

On windows open a command window and type ./startup.cmd

# Chapter 2

# BASELINE TRANSLATION

## 2.1 PROMPT GENERATION

For the translation prompt generation stages we work with 3 reference files

1. The markdown manuscript
2. The style guide
3. The language combination translation template

The Bash/CMD script checks the manuscript and sets up the config.json file with the correct manuscript name and chapter count. Then it runs the python file scripts/translate_prompt.py which will extract chapters from the manuscript and generate one prompt per chapter containing the chapter text, style guide text and template specifics.

## 2.2 CONSECUTIVE AI SUBMISSION

The script then runs a further python script to loop through all prompt files and submit them to the AI in groups of consecutive execution. The AI responses are then saved to text files and these are parsed to seperate out the translator notes from the translated text. Finally the translated texts are merged back into a single document "translated.md"

# Chapter 3

# BILINGUAL REVIEW/ACCURACY CHECKING

## 3.1 PROMPT GENERATION

For the accuracy check prompt generation stages we work with 4 reference files

1. The markdown manuscript
2. The baseline translation
3. The style guide
4. The language accuracy checking template

Then runs the python file scripts/AccCheck_prompt.py which will extract chapters from the manuscript and translation and generate one prompt per chapter containing the chapter text, style guide text and template specifics.

## 3.2 CONSECUTIVE AI SUBMISSION

The script then runs a further python script to loop through all prompt files and submit them to the AI in groups of consecutive execution. The AI responses are then saved to text files and these are parsed to seperate out the accuracy check notes from the corrected text. Finally the corrected texts are merged back into a single document "checked.md"

# Chapter 4

# LINE EDITING

## 4.1 PROMPT GENERATION

For the accuracy check prompt generation stages we work with 3 reference files

1. The checked text
2. The style guide
3. The language line editing template

Then runs the python file scripts/LineEdit_prompt.py which will extract chapters from the manuscript and translation and generate one prompt per chapter containing the chapter text, style guide text and template specifics.

## 4.2 CONSECUTIVE AI SUBMISSION

The script then runs a further python script to loop through all prompt files and submit them to the AI in groups of consecutive execution. The AI responses are then saved to text files and these are parsed to seperate out the accuracy check notes from the edited text. Finally the edited texts are merged back into a single document "LineEdited.md"

# Chapter 5

# RESULTS

The line edited document is then converted to word .docx format and saved into the top level directory along with the collected log files containing translating, accuracy checking and editing notes log files.

The individual results are all retained and the python scripts kept as individual modules so that single chapters can be revised as required then resubmitted to the AI and parsed/merged as required. To rerun a particular chapter through the AI delete the corresponding AI_response file and only that chapter will be redone. The individual scripts can be run from the command line if only one or two stages need repeating.

# Chapter 6

# ADDING PROMPT CACHING

The method outlined above sends the unchanging style guide and templates to the AI every chapter which incurs multiple tokenization costs. By splitting the prompts into static ”system” instructions and dynamic ”user” instructions and enabling prompt caching the tokens can be read from cache rather than repeatedly generated. Separate versions of the prompt generation scripts, the AI submission script and the command files (startup_ cache.cmd and startup_ cache.sh) have been added. For a typical 70k word English-German translation this saves approximately 35% API cost.